

Operating Systems (CSE2430)

Lab 4: The Buddy Allocator

J  r  mie Decouchant

Sign-off: 14 March 2025

In this session, you will learn and implement a simplified version of the buddy allocator.

The Buddy Allocator

Overview

This section is heavily inspired from a blog post¹ that we modified to fit our goals. In particular, the data structures that we manipulate are not completely identical. The job of a memory allocator is to manage a block of contiguous memory and chop it up into smaller pieces for individual allocations:

```
void *A = malloc(10);  
void *B = malloc(100);  
void *C = malloc(20);
```

```
-----  
|  A  | free |  B  |  C  |                free                |  
-----
```

Our allocator can allocate regions of size larger than $\text{MIN_ALLOC} = 2^{16}$ and smaller than $\text{MAX_ALLOC} = 2^{20}$. The starting address of the address range for the allocator is given by a pointer variable `base_ptr`. Every returned allocation will be an offset of this pointer within 0 (incl.) and MAX_ALLOC (excl.).

```
void *base_ptr;
```

The allocator needs to be fast at serving an allocation request, i.e. finding a suitable piece of free memory. It also needs to be fast at freeing memory, i.e. making a previously used piece of memory available for new allocations.

More specifically it implements two functions:

```
uint8_t *allocator_malloc(size_t requested_size);  
void allocator_free(void *ptr);
```

¹<https://bitsquid.blogspot.com/2015/08/allocation-adventures-3-buddy-allocator.html>

The `allocator_malloc` function allocates a region of size `requested_size` Bytes and returns a pointer to it if the allocation is successful or `NULL` otherwise. The `allocator_free` function takes a pointer and frees the allocated memory region that corresponds to it if it exists.

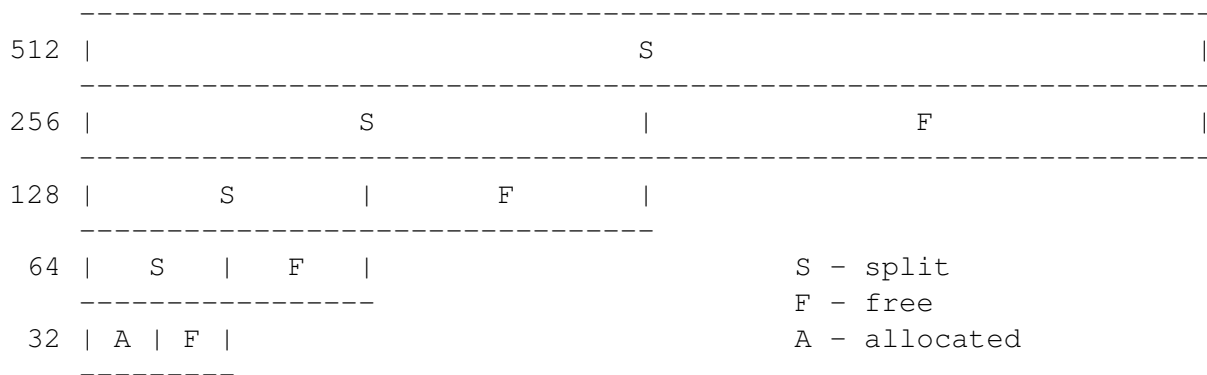
The buddy allocator works by repeatedly splitting memory blocks in half to create two smaller "buddies" until we get a block of the desired size.

If we start with a 512 KB block allocated from the OS, we can split it to create two 256 KB buddies. We can then take one of those and split it further into two 128 KB buddies, and so on.

When allocating, we check to see if we have a free block of the appropriate size. If not, we split a larger block as many times as necessary to get a block of a suitable size. So if we want 32 KB, we split the 128 KB block into 64 KB and then split one of those into 32 KB.

At the end of this, the state of the allocator will look something like this:

Buddy allocator after a single 32 KB allocation:



As you can see, this method of splitting means that the block sizes will always be a power of two. If you try to allocate something smaller, say 13 KB, the allocation will be rounded up to the nearest power of two (16 KB) and then get assigned a 16 KB block. So there is a significant amount of fragmentation happening here. This kind of fragmentation is called internal fragmentation since it is wasted memory inside a block, not wasted space between the blocks.

Merging in the buddy allocator is dead simple. Whenever a block is freed, we check if its buddy is also free. If it is, we merge the two buddies back together into the single block they were once split from. We continue to do this recursively, so if this newly created free block also has a free buddy, they get merged together into an even bigger block, etc.

The buddy allocator is pretty good at preventing external fragmentation, since whenever something is freed there is a pretty good chance that we can merge, and if we can't the "hole" should be filled pretty soon by a similarly sized allocation.

Linked lists of free blocks

To find a free block of a requested size: we put the free blocks of each size in an implicit linked list. To find a free block we just take the first item from the list of blocks of that size, remove it from the list and return it. If there is no block of the right size, we take the block of the next higher size and split it. We then use one of the two blocks we get and put the other one on the free list for that size. If the list of blocks of the bigger size is also empty, we can go to the even bigger size, etc.

To make things easier for us, let's introduce the concept of **levels**. We say that the single block that we start with, representing the entire buffer, is at level 0. When we split that we get two blocks at level 1. Splitting them, we get to level 2, etc. We also say that level i is a depth i .

```

if the list of free blocks at level n is empty
    allocate a block at level n-1 (using this algorithm)
    split the block into two blocks at level n
    insert the two blocks into the list of free blocks for level n
remove the first block from the list at level n and return it

```

```

/*
 * A doubly-chained list of free blocks per depth
 * Depth = 0, block size = MAX_ALLOC
 * From one depth to the next the block size is divided by 2, until
 * Depth = NUM_LEVELS-1, block_size = MIN_ALLOC
 */
block_list levels[NUM_LEVELS];

```

```
typedef struct block_list {
    struct block_list *prev;
    struct block_list *next;
} block_list;
```

```
void list_push(block_list *list, block_list *entry);
void list_remove(block_list *entry);
block_list *list_pop(block_list *list);
```

Block indices

512	0																													
256	1															2														
128	3					4					5					6														
64	7		8		9		10		11		12		13		14															
32	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30														

3

Merge index

The tricky part of the buddy allocator is to decide whether two blocks should be merged. Merging two blocks is simple, we just take the two blocks, remove them from the free list at level n and insert the merged block into the free list at level $n - 1$. The tricky part is to know when we should merge, i.e. when we are freeing a block p , how do we know if its buddy is also free, so that we can merge them?

First, note that we can easily compute the address of the buddy. Suppose we have freed a block p at level n . We can compute the index of that in the level as:

```
index_in_level_of(p,n) == (p - _buffer_start) / size_of_level(n)
```

If the index i is even, then the buddy is at index $i + 1$ and otherwise the buddy is at $i - 1$ and we can use the formula above to solve for the pointer, given the index.

So given the address of the buddy, let's call it `buddy_ptr`, how can we know if it is free or not? We could look through the free list for level n . If we find it there we know it is free and otherwise it's not. But there could be thousands of blocks and walking the list is hard on the cache.

To do better, we need to store some kind of extra information. What we can do is to store a bit for each block, telling us if that block is free or allocated. We can use the block index as described above to access this bitfield. This will require a total of $(1 \ll \text{num_level})$ bits.

But in fact we can do even better. We can get by with just half a bit per block: for each pair of buddies A and B we store the single bit `is_A_free XOR is_B_free`. We can easily maintain the state of that bit by flipping it each time one of the buddies is freed or allocated. When we consider making a merge we know that one of buddies is free, because it is only when a block has just been freed that we consider a merge. This means we can find out the state of the other block from the XORed bit. If it is 0, then both blocks are free. If it is 1 then it is just our block that is free.

So we can get by with just **one bit for every pair of block buddies**, that's half a bit per block. We will store this information in a bit array:

```
uint8_t *merge_index;
// indexed by buddy_pair_unique_id
// used to decide whether two buddies should be merged
```

Split index

All this time we have assumed that we know the level of the block that we are freeing. Otherwise we cannot compute the address of the buddy or its index in the block tree.

But to know the level of `ptr` we must know the size of its allocated block. Instead of storing the size explicitly, we can use the block index and store a single bit to keep track of whether the block at that level has been split or not.

To find the level of an allocated block we can then use the algorithm:

```
n = num_levels - 1
while n > 0
    if block_has_been_split(ptr, n-1)
        return n
    n = n - 1
return 0
```

Since the smallest blocks cannot be split, we only need $1 \ll (num_levels - 1)$ entries in the split index. This means that the cost of the split index is the same as for the merge index, 0.5 bits per block overall. It's a bit amazing that we can do all this with a total overhead of just 1 bit per block. To implement this functionality, we will store **a bit per possibly split block**:

```
uint8_t *is_split_index;
// indexed by global_id (0 to last index in last level-1)
// used to decide on the size of an allocated block based on its ptr
```

Assignments

For the assignments all code has to be written in C. Only the code has to be submitted to Weblab (more details on Weblab per assignment).

Please read the assignments carefully and make sure your code exactly implements the given descriptions. We will assess both the functionality and the clarity of your code and comments. **Weblab spec tests do not guarantee that your solution is fully correct.**

1.1

You are given a library file `library.c` that contains the following variables:

```
#define MAX_ALLOC_LOG2 20
#define MAX_ALLOC ((size_t) 1 << MAX_ALLOC_LOG2)
#define MIN_ALLOC_LOG2 16
#define MIN_ALLOC ((size_t) 1 << MIN_ALLOC_LOG2)
#define NUM_LEVELS (MAX_ALLOC_LOG2 - MIN_ALLOC_LOG2 + 1)

/*
 * Starting address of the address range for this allocator.
 * Every returned allocation will be an offset of this pointer
 * from 0 (incl.) to MAX_ALLOC (excl.)
 */
uint8_t *base_ptr;
```

Implement the following utility functions:

1. `int max_allocator_depth()`: returns the maximum depth, which equals the number of levels minus one.
2. `int min_global_id_at_depth(size_t depth)`: returns the smallest global block ID at a given depth.
3. `int max_global_id_at_depth(size_t depth)`: returns the largest global block ID at a given depth.
4. `int max_global_id()`: returns the largest global block ID across all depths.
5. `int parent_global_id(int global_id)`: returns the global block ID of a block's parent block.
6. `int left_child_global_id(int global_id)`: returns the global block ID of a block's left child.
7. `int right_child_global_id(int global_id)`: returns the global block ID of a block's right child.

8. `int level_index_to_global_id(int level_index, int depth)`: returns the global block ID of a block given its depth in the allocator and its position in the corresponding level.
9. `int global_id_to_depth(int global_id)`: returns the depth of a block given its global block ID.
10. `int global_id_to_level_index(int global_id)`: returns the position of a block in its level given its global block ID.
11. `uint8_t *global_id_to_ptr(int global_id)`: returns a pointer to a block given its global block ID. For example, `global_id_to_ptr(0)` should return a pointer equals to `base_ptr`. Given the size of our allocator, `uint8_t` is sufficient, while it would not be the case for larger ones.
12. `int ptr_to_global_id(void *ptr, size_t depth)`: returns the global block ID of a block given a pointer to it.
13. `int buddy_global_id(int global_id)`: returns the global block ID of a block's buddy.
14. `int max_buddy_pair_unique_id()`: returns the largest possible buddy ID of a block.
15. `int buddy_pair_unique_id(int global_id)`: returns the buddy pair ID of a block given its global block ID. Note that two buddies have the same buddy pair ID. Buddy pair IDs range from 0 (for block 0 only) to a maximum value and include all intermediary values. For example, blocks 1 and 2 have buddy pair ID 1, blocks 3 and 4 have buddy pair ID 2, blocks 5 and 6 have buddy pair ID 3, etc.
16. `size_t block_size_at_depth(size_t depth)`: returns the size of blocks at a given depth.
17. `int num_blocks_at_depth(size_t depth)`: returns the number of blocks at a given depth.
18. `int depth_for_request(size_t requested_size)`: returns the largest depth at which blocks are larger than a given size.

Note that some of these functions can be implemented using bitwise operations.

1.2

Implement the following functions that manipulate the split index:

1. `uint8_t block_is_split(int global_id)`: returns a boolean (0 or 1 only) that indicates whether a block is split given its global block ID. Note that this function does not return `is_split_index[global_id]`. Instead, its answer depends on the `global_idth` bit in `is_split_index`. For example, knowing whether block 9 is split requires looking at the 9th bit in `is_split_index`, which happens to be located inside `is_split_index[1]`.
2. `uint8_t parent_is_split(int global_id)`: returns a boolean (0 or 1 only) that indicates whether a block's parent is split given its global block ID.
3. `void flip_parent_is_split(int global_id)`: flip the bit in `is_split_index` that corresponds to the parent of a block whose block global ID is given.
4. `int depth_allocated_ptr(void *ptr)`: returns the depth of an allocated block given a pointer to it. Note that this function uses the split index and that its pseudocode has been given in its description.

Implement the following functions that manipulate the merge index:

1. `void flip_block_is_free(int global_id)`: flips the bit that corresponds to a global block ID (i.e., to its buddy pair ID) in the merge index.

2. `uint8_t block_is_free(int global_id)`: returns a boolean (0 or 1 only) that returns the bit that corresponds to a global block ID (i.e., to its buddy pair ID) in the merge index.

1.3

Implement the following functions that manipulate both the split index and the merge index:

1. `uint8_t *allocator_malloc(size_t requested_size)`
2. `void allocator_free(void *ptr)`

The first function, `allocator_malloc(size_t requested_size)`, returns a pointer to a block that is just large enough to contain the requested size. This function should first find the smallest free block that is larger than `requested_size`. Afterwards, this block should be splitted, if necessary, and one of its children returned while the others would become free blocks. Note that it is also necessary to flip the adequate bits in the merge index and in the split index across this function.

The second function, `void allocator_free(void *ptr)`, frees a block starting from a pointer to it. As a block is freed, it might get coalesced with a free buddy. Note that coalescing might happen over multiple levels. Here again, it is necessary to flip the adequate bits in the merge index and in the split index across this function.

Note that you are provided with the following functions:

- `void init_allocator()`:
- `void free_allocator()`:
- `void print_block_states()`: a function that prints a representation of the allocator's state using a letter per block. An allocated block uses letter A, a split block uses letter S, and a free block uses letter F.

You might find the last function useful to debug your code. Please tell us if you detect a bug in this function, this might be the most complex function of this project. However, note that debugging your code can also be done by directly looking into the values stored inside the split index and inside the merge index, which is a less visual but more reliable method.